



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
GRADO EN INGENIERÍA INFORMÁTICA

Trabajo de Fin de Grado

IUCE Reservas: Sistema web de gestión de reservas de espacios académicos

Presentado por
Enrique González Gutiérrez

Tutores
José Ignacio Santos Martín
David González Asenjo

Curso 2025-2026
Burgos, junio de 2026

D. José Ignacio Santos Martín, profesor del departamento de Ingeniería de Organización, área de Organización de Empresas.

Expone:

Que el alumno D. Enrique González Gutiérrez, con DNI 72220205K, ha realizado el Trabajo Final de Grado en Ingeniería Informática titulado «IUCE Reservas: Sistema web de gestión de reservas de espacios académicos».

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección del que suscribe, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, junio de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. José Ignacio Santos Martín

D. David González Asenjo

RESUMEN

El Instituto Universitario de Ciencias de la Educación (IUCE) de la Universidad de Salamanca gestiona la reserva de sus aulas, laboratorios y salas de uso múltiple mediante un proceso manual basado en correo electrónico y hojas de cálculo, lo que ocasiona retrasos, solapamientos en las asignaciones y una sobrecarga administrativa creciente. Este Trabajo de Fin de Grado presenta IUCE Reservas, una aplicación web a medida que digitaliza por completo el flujo de solicitud, aprobación y consulta de reservas para el personal investigador, profesorado y personal técnico del instituto.

El sistema implementa un catálogo en línea de los espacios disponibles, un formulario de solicitud con validación automática de solapamientos, un panel de aprobación administrativa para el personal autorizado, notificaciones por correo electrónico en cada cambio de estado y una visualización en calendario de las reservas confirmadas.

El desarrollo se ha planificado y ejecutado aplicando la metodología Scrum a lo largo de diez sprints, organizados en torno a doce épicas, con un Product Backlog gestionado de forma trazable y revisado al cierre de cada iteración. El resultado, al término del desarrollo, es una plataforma desplegable en producción y accesible para toda la comunidad del IUCE en el dominio reservas.iuce.usal.es. La calidad del código se asegura de forma continua mediante una suite de pruebas unitarias automatizadas con Vitest y un análisis estático con SonarCloud integrado en el flujo de integración continua.

Palabras clave: Next.js, Prisma, PostgreSQL, gestión de reservas, Scrum, autenticación passwordless, despliegue Node.js.

ABSTRACT

The Institute of Education Sciences (IUCE) at the University of Salamanca manages the booking of its classrooms, laboratories and multi-purpose rooms through a manual process based on email and spreadsheets, which causes delays, overlapping assignments and growing administrative burden. This Bachelor's Thesis presents IUCE Reservas, a custom web application that fully digitises the request, approval and consultation workflow for the institute's researchers, teaching staff and technical personnel.

The system provides an online catalogue of available spaces, a reservation request form with automatic overlap validation, an administrative approval panel for authorised staff, email notifications at every state change and a calendar view of confirmed reservations.

Development has been planned and executed applying the Scrum methodology over ten sprints, organised around twelve epics, with a fully traceable Product Backlog reviewed at the end of each iteration. By the end of the development, the result is a platform ready to be deployed in production and accessible to the entire IUCE community at the domain `reservas.iuce.usal.es`. Code quality is continuously assured through an automated unit-test suite with Vitest and static analysis with SonarCloud integrated into the continuous integration pipeline.

Keywords: Next.js, Prisma, PostgreSQL, room booking, Scrum, passwordless authentication, Node.js deployment.

ÍNDICE

1. Introducción	3
2. Objetivos del proyecto	5
2.1. Objetivo general	5
2.2. Objetivos específicos del producto	5
2.3. Objetivos específicos del proceso	6
3. Conceptos teóricos	7
3.1. Aplicaciones web y arquitectura cliente-servidor	7
3.2. Modelo relacional y mapeo objeto-relacional	8
3.3. Autenticación passwordless mediante magic links	9
3.4. Gestión de reservas como problema temporal	10
3.5. Métodos ágiles aplicados al proyecto.....	11
4. Técnicas y herramientas	13
4.1. Metodología empleada	13
4.2. Alternativas estudiadas.....	14
4.3. Análisis comparativo del framework web: Next.js frente a HTML/CSS/JS clásico y a React con backend separado	15
4.4. Autenticación: magic link frente a SSO institucional y OAuth con Google.....	15
4.5. Herramientas y tecnologías elegidas	16
5. Aspectos relevantes del desarrollo del proyecto	18
5.1. Ciclo de vida y planificación adaptativa	18
5.2. Modelo de datos: decisiones de diseño	19
5.3. Máquina de estados de la reserva.....	20
5.4. Autenticación passwordless con NextAuth.....	22
5.5. Validación de solapamientos y concurrencia	22
5.6. Arquitectura del frontend: App Router y Server Components.....	24

5.7. Aseguramiento de calidad	25
5.8. Panel de métricas y analítica de uso.....	25
5.9. Rendimiento y consumo de recursos.....	26
5.10. Ingeniería de software como marco del proceso	27
5.11. El papel de la IA generativa en el proyecto	28
6. Trabajos relacionados.....	31
7. Conclusiones y líneas de trabajo futuras	32
7.1. Conclusiones	32
7.2. Líneas de trabajo futuras	33
8. Referencias bibliográficas	35
Anexos.....	37
Anexo A. Plan de Proyecto Software.....	37
Anexo B. Especificación de Requisitos	37
Anexo C. Especificación de diseño.....	37
Anexo D. Documentación técnica de programación.....	38
Releases y entregables del proyecto.....	38
Anexo E. Documentación de usuario.....	38
Manual de personal de administración.....	38
Anexo F. Anexo de sostenibilización curricular	39

ÍNDICE DE FIGURAS

Figura 1. Arquitectura de despliegue del sistema de producción.....	17
Figura 2. Cronograma de releases y sprints.	19

ÍNDICE DE TABLAS

Tabla 1. Alternativas estudiadas para los componentes principales del sistema.....	14
---	----

1. INTRODUCCIÓN

El Instituto Universitario de Ciencias de la Educación (IUCE) es un centro propio de la Universidad de Salamanca dedicado a la investigación, formación e innovación en el ámbito de las ciencias de la educación. El IUCE acoge grupos de investigación, programas de doctorado, seminarios formativos y actividades abiertas tanto a personal académico como a profesionales externos vinculados al ámbito educativo. Para soportar toda esta actividad, el instituto dispone de un conjunto de espacios físicos (aulas docentes, laboratorios y salas de usos múltiples) que son utilizados de manera intensiva y compartida entre los distintos colectivos del centro a lo largo del curso académico.

La gestión actual de la reserva de estos espacios sigue un procedimiento puramente manual: el personal docente e investigador interesado solicita la disponibilidad por correo electrónico al equipo administrativo del instituto, que comprueba en una hoja de cálculo compartida si el horario solicitado está libre y, en caso afirmativo, anota la nueva reserva. Este flujo, perfectamente válido cuando el volumen de reservas era reducido, se ha convertido progresivamente en un cuello de botella: respuestas con varios días de retraso, errores de transcripción que provocan solapamientos entre reservas, ausencia de notificaciones automáticas al solicitante y una sobrecarga de trabajo administrativo que crece con cada nuevo curso. Esta situación, además de generar fricción operativa, dificulta que el propio personal del instituto pueda planificar con autonomía sus actividades.

Este Trabajo de Fin de Grado parte precisamente de esa oportunidad: la de digitalizar de manera completa el proceso de reserva mediante una aplicación web institucional, IUCE Reservas, que sustituya el flujo basado en correo y hoja de cálculo por una plataforma centralizada accesible desde cualquier navegador. La aplicación permite a cualquier miembro del IUCE consultar el catálogo de espacios disponibles, solicitar una reserva indicando título, fecha, franja horaria y número de asistentes, recibir notificaciones automáticas en cada cambio de estado de su solicitud y consultar en cualquier momento el estado de sus reservas. Por su parte, el personal autorizado dispone de un panel de aprobación administrativa que centraliza todas las solicitudes pendientes y permite aprobarlas o rechazarlas con la información necesaria para tomar la decisión. La autenticación se realiza contra las cuentas institucionales @usal.es ya existentes, evitando la creación de usuarios y contraseñas adicionales.

Más allá del producto resultante, este trabajo plantea un objetivo de carácter académico igualmente relevante: aplicar de forma rigurosa la metodología Scrum a lo largo de todo el ciclo de desarrollo. El proyecto se ha organizado en diez sprints sucesivos, agrupados en doce épicas que cubren desde la fase previa de análisis y diseño hasta las distintas iteraciones de implementación, despliegue y validación. Cada incremento se ha integrado, revisado y desplegado de forma continua, dando lugar, al cierre del proyecto, a una versión funcional del sistema accesible en producción para toda la comunidad del IUCE en el dominio reservas.iuce.usal.es.

La presente memoria se estructura en siete capítulos que reflejan el recorrido natural de un proyecto software completo. El capítulo 2 detalla los objetivos funcionales y técnicos perseguidos por el trabajo. El capítulo 3 sintetiza los conceptos teóricos necesarios para comprender el dominio del problema y las decisiones de diseño tomadas. El capítulo 4 presenta la metodología empleada y las herramientas seleccionadas, junto con las alternativas estudiadas y descartadas. El capítulo 5, núcleo de la memoria, recoge los aspectos más relevantes del análisis, diseño, implementación y despliegue del sistema. El capítulo 6 contextualiza la solución frente a otros sistemas similares, comerciales y de código abierto. Por último, el capítulo 7 recoge las conclusiones extraídas del desarrollo y propone líneas de trabajo futuras. La memoria se acompaña de varios anexos con material complementario y del repositorio público con el código fuente del proyecto.

2. OBJETIVOS DEL PROYECTO

Este apartado describe el objetivo principal que persigue el desarrollo de IUCE Reservas y lo desglosa en dos conjuntos de objetivos específicos: los objetivos funcionales, asociados a las capacidades que el sistema debe ofrecer al usuario final, y los objetivos técnicos, vinculados al rigor metodológico y a las prácticas de ingeniería del software aplicadas durante el desarrollo.

2.1. Objetivo general

El objetivo general del proyecto es diseñar, desarrollar y desplegar en producción una aplicación web que digitalice por completo la gestión de reservas de los espacios físicos del Instituto Universitario de Ciencias de la Educación, sustituyendo el flujo manual basado en correo electrónico y hojas de cálculo por una plataforma centralizada que permita al personal del instituto solicitar reservas y al equipo administrativo aprobarlas o rechazarlas de forma trazable, segura y eficiente.

2.2. Objetivos específicos del producto

Los siguientes objetivos definen el alcance del producto desde la perspectiva del usuario final del sistema:

- Disponer de un catálogo de los espacios reservables, accesible para los usuarios autenticados, con la información relevante de cada uno: nombre, código, capacidad, planta, equipamiento disponible y accesibilidad.
- Permitir la autenticación institucional del personal del IUCE mediante las cuentas @usal.es ya existentes, sin necesidad de gestionar nuevas credenciales.
- Ofrecer un formulario de solicitud de reserva que recoja título, descripción, fecha, franja horaria y número de asistentes, validando automáticamente que la solicitud no se solape con reservas previas y que respete la capacidad del espacio.
- Implementar un panel de administración que centralice todas las solicitudes pendientes y permita al personal autorizado aprobarlas o rechazarlas con la información necesaria para tomar la decisión.
- Enviar notificaciones automáticas por correo electrónico al solicitante en cada cambio de estado de su reserva, así como al personal administrativo cuando se registre una nueva solicitud. El aviso de nueva solicitud al personal administrativo es configurable por cada administrador mediante una preferencia en su perfil (receiveAdminEmails): por defecto se recibe, pero un administrador puede silenciarlo sin perder su rol, de modo que solo se notifica a los administradores que tienen la preferencia activada.
- Mostrar a cada usuario el estado actual de sus reservas (pendientes, aprobadas y rechazadas) y permitir su consulta histórica.

- Proporcionar una visualización en calendario de las reservas confirmadas para facilitar la planificación de actividades.
- Ofrecer un flujo accesible desde cualquier navegador, sin instalación adicional, y compatible con dispositivos de escritorio.

2.3. Objetivos específicos del proceso

Estos objetivos atienden al rigor metodológico y a las buenas prácticas de ingeniería que se persiguen como ejercicio académico durante la realización del trabajo:

- Aplicar Scrum de forma adaptada al contexto de un Trabajo de Fin de Grado individual, organizado en diez sprints y doce épicas, con un Product Backlog vivo, planificación al inicio y retrospectiva al cierre de cada iteración.
- Mantener una trazabilidad completa entre las historias de usuario del backlog, los issues del repositorio y los commits que las implementan, así como entre las tarjetas del tablero de Zube y las issues de GitHub mediante una sincronización bidireccional, de forma que cualquier cambio en el código sea siempre justificable a partir de su user story de origen.
- Construir el sistema sobre tecnologías modernas y de código abierto, asegurando la mantenibilidad a largo plazo del proyecto y su transferibilidad a otras unidades académicas con necesidades similares.
- Garantizar la calidad del código mediante revisión continua, conventional commits, una cobertura razonable de pruebas unitarias automatizadas y un proceso de integración reproducible.
- Integrar el repositorio con una plataforma de análisis estático (SonarCloud) que evalúe seguridad, fiabilidad y mantenibilidad en cada push a la rama principal, y mantener un Quality Gate verde como condición a lo largo de todo el ciclo de desarrollo.
- Publicar releases versionadas en GitHub al cierre de los hitos relevantes del proyecto, acompañadas de changelog, evidencias de calidad e instrucciones de despliegue local, garantizando la trazabilidad de cada incremento entregado.
- Empaquetar y desplegar la aplicación en producción en un servidor del propio instituto, con certificado SSL gestionado automáticamente, dejando la aplicación operativa, accesible y mantenible más allá de la entrega académica del trabajo.
- Documentar las decisiones técnicas más relevantes en la propia memoria, de forma que sirva como referencia tanto para el tribunal como para futuros responsables del mantenimiento del sistema.

3. CONCEPTOS TEÓRICOS

Antes de presentar las decisiones de diseño e implementación, conviene fijar los conceptos teóricos sobre los que se apoya el sistema desarrollado en este Trabajo de Fin de Grado, IUCE Reservas. Este capítulo se centra en las nociones generales que el lector necesita para entender el dominio del problema, la naturaleza de las entidades manipuladas por el sistema y la metodología seguida durante el desarrollo. Las tecnologías concretas empleadas se abordan más adelante en el capítulo 4. La exposición se organiza en cinco bloques: el modelo de aplicación web, el paradigma relacional y su mapeo a objetos, la autenticación sin contraseña, la modelización de una reserva como objeto temporal y la metodología ágil Scrum.

3.1. Aplicaciones web y arquitectura cliente-servidor

Una aplicación web es un programa cuya interfaz de usuario se ejecuta en un navegador y cuya lógica reside, total o parcialmente, en un servidor remoto al que se accede mediante el protocolo HTTP. Como describen Fielding y Reschke (2014, §2.1) en la especificación de HTTP/1.1, la comunicación se basa en peticiones y respuestas sin estado, de modo que cada interacción del cliente con el servidor es independiente de las anteriores. Esa independencia obliga a introducir mecanismos adicionales (cookies, tokens o sesiones) cuando se necesita preservar información entre peticiones consecutivas, una restricción que afecta tanto al diseño de la autenticación como al modelado del estado de una reserva.

El modelo cliente-servidor separa responsabilidades en dos componentes diferenciados. El cliente, que en este caso es el navegador del usuario, se encarga de la presentación y de la captura de eventos. El servidor concentra la lógica de negocio y el acceso a la base de datos. Sommerville (2016) describe esta arquitectura como una estructura en capas, donde las decisiones de diseño que afectan a la calidad del software (seguridad, integridad y persistencia) deben situarse en la capa de servidor, ya que es la única zona del sistema que el desarrollador controla por completo. En IUCE Reservas, esta separación justifica que toda comprobación crítica, como la validación de solapamientos o la autorización por rol, se realice en el servidor, con independencia de las verificaciones cosméticas que pueda añadir la interfaz.

Las aplicaciones web actuales heredan, además del modelo de comunicación HTTP, un conjunto de principios arquitectónicos formulados por Fielding (2000) en su tesis doctoral, bajo la denominación REST (Representational State Transfer). Sus restricciones fundamentales son cinco: cliente-servidor, ausencia de estado en el servidor, posibilidad de cacheo, interfaz uniforme y estructura en capas. Cuando se respetan, conducen a sistemas escalables y razonables de mantener a largo plazo. La interfaz uniforme propone identificar cada recurso mediante una URI estable y manipularlo a través de un conjunto pequeño y bien definido de operaciones, normalmente las primitivas de HTTP, sobre representaciones intercambiables.

Esta uniformidad reduce el coste cognitivo de aprender un nuevo sistema y favorece la reutilización de infraestructura genérica como cachés, proxies inversos o servidores de aplicaciones.

El estado de la sesión genera una tensión recurrente con la naturaleza sin estado de HTTP. Hay dos estrategias principales para preservar información entre peticiones. La primera utiliza sesiones del lado del servidor: el cliente recibe únicamente un identificador opaco y el estado real reside en una base de datos o caché del servidor. La segunda recurre a tokens autocontenidos, normalmente firmados criptográficamente, que viajan en cada petición y permiten al servidor reconstruir la identidad del usuario sin consulta adicional. Cada enfoque tiene sus contrapartidas en escalabilidad, capacidad para revocar credenciales y complejidad operativa. Esta tensión condiciona directamente cómo se diseña la autenticación, materia que se desarrolla en el apartado dedicado al modelo passwordless adoptado en este proyecto.

3.2. Modelo relacional y mapeo objeto-relacional

El modelo relacional, formulado por Codd (1970, p. 379) en su artículo fundacional, organiza los datos en relaciones (tablas) compuestas por tuplas (filas) y atributos (columnas), y establece la integridad referencial mediante claves primarias y foráneas. Sigue siendo el modelo más extendido en sistemas de información por su solidez teórica, su capacidad de garantizar consistencia mediante transacciones ACID y la madurez del lenguaje SQL. En el dominio de las reservas, las relaciones entre usuario, espacio y reserva se expresan de forma natural mediante claves foráneas, y restricciones como la unicidad o el rango de fechas pueden delegarse al motor de base de datos.

El acceso a una base de datos relacional desde un lenguaje orientado a objetos plantea la conocida *impedance mismatch*: las tablas no son objetos y las relaciones no son referencias en memoria. Un mapeador objeto-relacional (ORM) resuelve este desajuste traduciendo automáticamente entre filas de la base de datos y objetos del programa. Fowler (2002) define este patrón como una capa intermedia que oculta el SQL al desarrollador y permite trabajar con un modelo de dominio expresivo sin renunciar a las garantías del motor relacional. En la práctica, el código de negocio puede manipular reservas como objetos, mientras que las consultas, las migraciones de esquema y las restricciones de integridad quedan declaradas en un único punto.

La integridad de los datos en un sistema relacional descansa sobre tres clases de restricciones que el motor verifica de forma automática. Las restricciones de dominio limitan los valores admisibles en cada columna. Las de clave aseguran la unicidad y la identificabilidad de cada tupla. Las de integridad referencial vinculan las claves foráneas con las claves primarias correspondientes en otras relaciones. Date (2003) destaca que esta separación entre la definición declarativa de las restricciones y su comprobación en tiempo de ejecución es una de las contribuciones más prácticas del modelo: el desarrollador especifica qué condiciones deben

cumplirse y el motor garantiza que ninguna operación deje la base de datos en un estado que las viole. En el dominio de las reservas, esta capacidad permite delegar al SGBD comprobaciones tan diversas como la existencia del espacio reservado, el rango admisible de fechas o la unicidad del identificador de cada registro, sin que la lógica de aplicación tenga que duplicar estas verificaciones.

El diseño de un esquema relacional pasa habitualmente por un proceso de normalización destinado a eliminar redundancias y prevenir anomalías de actualización. Aplicado con criterio, este proceso conduce a un conjunto reducido de tablas en las que cada hecho se almacena en un único lugar; aplicado en exceso, puede degradar el rendimiento al multiplicar las uniones (joins) requeridas para reconstruir información de uso frecuente. La práctica habitual en sistemas de información de tamaño medio consiste en partir de un esquema en tercera forma normal y desnormalizar selectivamente solo cuando una medición de rendimiento lo justifique. El esquema utilizado en este proyecto sigue esta línea: cada entidad del dominio (usuario, espacio, reserva y bloqueo) ocupa una única tabla, y las relaciones se expresan mediante claves foráneas, indexadas en los caminos de consulta principales, sin que ninguna optimización prematura altere el modelo conceptual.

3.3. Autenticación passwordless mediante magic links

La autenticación tradicional basada en par usuario-contraseña ha sido objeto de crítica sostenida desde hace más de una década. Bonneau y otros (2012, pp. 553-554) demostraron, en un estudio comparativo de esquemas de autenticación web, que las contraseñas presentan debilidades estructurales (reutilización entre servicios, susceptibilidad a phishing y baja entropía efectiva) que ningún esquema heredado consigue mitigar por completo. Como respuesta han surgido los esquemas passwordless, en los que el usuario no memoriza ningún secreto y el factor de autenticación se delega en un canal lateral previamente verificado, normalmente el correo electrónico institucional.

El mecanismo concreto empleado en este proyecto es el magic link. Cuando el usuario solicita iniciar sesión, el servidor genera un token aleatorio de un solo uso y vida útil reducida, lo asocia a la dirección de correo del usuario y se lo remite mediante un enlace único. Al hacer clic, el navegador devuelve el token al servidor, que lo invalida y emite una sesión autenticada. El patrón puede entenderse como una forma de autenticación basada en posesión, en la que la prueba de identidad consiste en demostrar el control del buzón de correo, en línea con el modelo de delegación que Hardt (2012) formaliza en el marco OAuth 2.0 para escenarios análogos de acceso autorizado. La principal ventaja en un entorno académico está en aprovechar una identidad institucional que la universidad ya gestiona, lo que evita un registro adicional y reduce la superficie de ataque al no almacenar credenciales.

Más que una contraseña efímera, el magic link materializa un patrón general descrito en la literatura de seguridad: separar el identificador del usuario y el factor de autenticación, y delegar

este último en un canal previamente verificado. Esta separación tiene dos consecuencias arquitectónicas relevantes. Por un lado, el sistema no necesita almacenar ni proteger un repositorio de hashes de contraseñas, lo que elimina toda una clase de vulnerabilidades (filtrado de bases de datos, ataques de fuerza bruta sobre hashes débiles, requisitos de rotación periódica) que han sido fuente recurrente de incidentes documentados. Por otro lado, el control de acceso al buzón se delega íntegramente en el proveedor institucional de correo, lo que en un entorno académico se alinea de forma natural con las políticas de identidad ya existentes en la universidad.

Desde la perspectiva del usuario, la autenticación passwordless mediante magic link reduce la fricción de acceso a una sola acción (introducir el correo institucional y abrir un enlace), sin sacrificar el segundo factor implícito que aporta el control sobre la cuenta de correo. Existen, no obstante, contrapartidas que el diseño del sistema debe contemplar: la dependencia operacional respecto al servicio de correo, la latencia añadida en la primera autenticación de cada sesión y la necesidad de gestionar con cuidado la vida útil del token y su comportamiento ante reusos o reintentos. La configuración elegida en este proyecto fija una vida útil corta para el enlace, invalida el token tras su primer uso y mantiene la sesión autenticada mediante una cookie firmada de duración acotada, equilibrando comodidad de uso y exposición frente a la captura del enlace.

3.4. Gestión de reservas como problema temporal

El problema central que resuelve IUCE Reservas es muy concreto: si un profesor reserva el aula 12 de 10:00 a 12:00 y otro intenta reservarla de 11:00 a 13:00, el sistema debe detectar el conflicto y rechazar la segunda solicitud. Cada reserva se representa internamente como un tramo horario con inicio y fin, asociado a un espacio y a un usuario, y la regla práctica para decidir si dos reservas chocan es directa: hay conflicto cuando una empieza antes de que termine la otra y termina después de que la otra haya empezado. Esta regla es la traducción de un resultado clásico que Allen (1983) formuló para el razonamiento sobre intervalos temporales y que se utiliza desde hace décadas en todo tipo de sistemas con calendarios y franjas horarias, en sistemas de planificación de quirófanos y en reservas en hoteles y empresas.

Una reserva, además, no es algo estático: pasa por varios estados a lo largo de su vida. Cuando un usuario la solicita queda pendiente, un administrador puede aprobarla o rechazarla, y el propio usuario puede cancelarla mientras siga vigente. No todas las transiciones son válidas (una reserva ya rechazada no puede aprobarse después, por ejemplo) y cada cambio de estado dispara efectos asociados, como enviar un correo al solicitante avisando del resultado. Toda esta lógica se representa de forma natural mediante una máquina de estados, una herramienta de modelado que Harel (1987) consolidó en ingeniería del software con su trabajo sobre statecharts. Mantener esa lógica concentrada en un único lugar del servidor, separada de la

interfaz, permite verificar con facilidad que ningún cambio de estado escape a las comprobaciones de permisos.

El caso de las reservas recurrentes añade una capa adicional de complejidad. Una clase que se imparte todos los lunes a la misma hora durante un cuatrimestre se traduce en quince o dieciséis reservas que comparten un patrón común, no en una única entrada de calendario. Cada instancia tiene que comprobarse de forma independiente frente al resto de reservas y bloqueos del calendario, ya que cualquiera de esos lunes puede caer en festivo o coincidir con otra actividad ya programada. En la práctica, esto deja dos formas de almacenarlas: guardar cada sesión como un registro independiente, o guardar solo la regla de recurrencia y generar las sesiones al consultarlas. La primera opción ocupa más espacio pero hace triviales las consultas y permite cancelar una sesión concreta sin afectar a las demás; IUCE Reservas adopta este enfoque por su mayor simplicidad y por encajar de forma directa con las vistas de calendario que ven los usuarios.

Un último aspecto importante, y que en la práctica es donde más fácilmente se cuelan errores, es lo que ocurre cuando dos personas intentan reservar exactamente la misma franja al mismo tiempo. Si la comprobación de solapamientos se hace solo en la capa de aplicación, existe una pequeña ventana de tiempo entre que el sistema consulta si la franja está libre y graba la nueva reserva; en ese intervalo, una segunda solicitud puede colarse y acabar con dos reservas pisándose. La forma fiable de resolver este caso es delegar el arbitraje en el motor de base de datos, que sí puede garantizar que la comprobación y el guardado se hagan como una operación indivisible. Por eso, la validación efectiva frente al doble solapamiento no se sitúa en una transacción atómica durante la creación, sino en la re-comprobación de disponibilidad que el endpoint de aprobación ejecuta frente a las reservas ya APPROVED. La comprobación de solapamientos en el momento de crear la reserva se realiza en la capa de aplicación, no dentro de una transacción. Envolver comprobación y escritura en una única transacción con aislamiento estricto se deja señalado como mejora futura (véase la sección 5.5).

3.5. Métodos ágiles aplicados al proyecto

El desarrollo de IUCE Reservas se rige por los principios de los métodos ágiles, tomando Scrum como marco teórico de referencia pero adaptándolo a las circunstancias reales de un Trabajo de Fin de Grado individual. Schwaber y Sutherland (2020) describen Scrum como un proceso empírico apoyado en tres pilares (transparencia, inspección y adaptación) y organizado en torno a tres responsabilidades (Product Owner, Scrum Master y Developers), tres artefactos y cinco eventos. Este marco está pensado para equipos multidisciplinares de varias personas; trasladarlo sin matices a un proyecto donde el autor es a la vez analista, diseñador, programador y responsable de despliegue resultaría artificial. La adaptación adoptada conserva los elementos del marco que aportan valor real (la cadencia iterativa, el backlog vivo, la revisión periódica

con quien hace el rol de cliente) y descarta los que no encajan en un proyecto unipersonal (los eventos diarios, los roles segregados, la estimación con story points compartida entre equipo).

De los tres artefactos canónicos de Scrum (Product Backlog, Sprint Backlog e Incremento), el proyecto adopta el Product Backlog como herramienta central, gestionado en Zube con sincronización bidireccional a GitHub Issues, y reconoce el Sprint Backlog implícitamente como el subconjunto de tarjetas comprometidas en cada iteración. El Incremento se materializa al cierre de cada sprint mediante el merge a la rama principal y, en hitos relevantes, mediante releases versionadas en GitHub. De los cinco eventos, se mantienen tres en versión adaptada: la planificación al inicio de cada sprint, la revisión al cierre (validando el incremento contra los criterios acordados) y una retrospectiva informal que ajusta el proceso para la siguiente iteración. Los Daily Scrums quedan descartados por innecesarios en un proyecto unipersonal, y la Sprint Planning formal con poker planning se sustituye por una estimación cualitativa propia del autor. Para gestionar la complejidad del backlog se recurre a dos abstracciones de uso extendido en la comunidad ágil: las épicas, que agrupan funcionalidades afines de gran tamaño, y las historias de usuario, formuladas por Cohn (2004) como descripciones breves de funcionalidad desde la perspectiva del usuario final.

Por encima de Scrum, el Manifiesto Ágil firmado por Beck y otros (2001) sienta los principios filosóficos sobre los que operan los métodos ágiles en su conjunto. Sus cuatro valores priorizan a las personas y sus interacciones sobre los procesos y herramientas, el software que funciona sobre la documentación exhaustiva, la colaboración con el cliente sobre la negociación contractual y la respuesta al cambio sobre el seguimiento de un plan rígido. Estos valores no eliminan la planificación ni la documentación; las relativizan frente a la finalidad última del desarrollo, que es entregar software útil de forma temprana y continua. El ciclo iterativo con sprints de duración acotada (entre cuatro y siete días en este proyecto) materializa esa filosofía: actúa como mecanismo de control que evita que cualquier error de criterio se prolongue más allá del horizonte del sprint sin ser detectado y corregido en la siguiente tutoría del TFG.

El backlog del producto se gestiona como una lista priorizada y refinada de forma continua, en la que cada elemento dispone de criterios de aceptación explícitos y una estimación cualitativa de tamaño. La planificación de cada sprint convierte un subconjunto del backlog en el compromiso del periodo, y el incremento resultante se valida al cierre. Una retrospectiva informal traslada después el objeto de inspección desde el producto hacia el propio proceso e identifica ajustes para la siguiente iteración. La denominación habitual de estas adaptaciones en la literatura es la de un Scrum reducido o pragmático, frecuente en contextos académicos donde aplicar el marco al completo resulta desproporcionado para la escala y el equipo del proyecto. En el contexto de un Trabajo de Fin de Grado, este ciclo cerrado de planificación, ejecución, revisión y mejora aporta dos garantías especialmente valiosas: una trazabilidad clara entre el avance del proyecto y los hitos académicos, y un mecanismo natural de defensa frente al riesgo de sobre ingeniería o de desviación respecto al alcance comprometido.

4. TÉCNICAS Y HERRAMIENTAS

Este capítulo describe cómo se ha llevado a cabo el desarrollo de IUCE Reservas: qué metodología ha guiado el trabajo, qué herramientas se han empleado para gestionarlo y, finalmente, qué piezas tecnológicas componen el sistema en su versión actual. Para las decisiones más relevantes se recoge también el conjunto de alternativas que se contemplaron y los criterios que llevaron a la elección final.

4.1. Metodología empleada

El desarrollo se ha organizado siguiendo Scrum, con los ajustes habituales para un proyecto unipersonal. El trabajo se planifica en sprints de aproximadamente una semana, ajustados a la cadencia de las reuniones periódicas con los tutores del TFG, que cumplen el papel combinado de Sprint Review y Sprint Planning del sprint siguiente. La Sprint Retrospective queda condensada en una breve revisión personal al cierre de cada semana, recogida en las notas de avance del repositorio.

El Product Backlog está estructurado en doce épicas, una previa de análisis y diseño (EPIC-00) y once temáticas de implementación que cubren cada gran área del sistema, desde la autenticación hasta el despliegue: EPIC-00 Análisis y Diseño, EPIC-01 Setup y Arquitectura, EPIC-02 Autenticación y Usuarios, EPIC-03 Gestión de Espacios, EPIC-04 Motor de Reservas, EPIC-05 Notificaciones por Email, EPIC-06 Calendario y Visualización, EPIC-07 Panel de Administración, EPIC-08 UX y Diseño, EPIC-09 Calidad y Testing, EPIC-10 DevOps y Despliegue, y EPIC-11 Releases y Distribución. Cada épica se descompone en historias de usuario formuladas en el formato clásico «como [rol], quiero [acción] para [beneficio]», con sus criterios de aceptación explícitos. La estimación se realiza en puntos de historia mediante una escala de Fibonacci modificada (1, 2, 3, 5, 8, 13), lo que permite expresar la complejidad relativa de cada tarea sin comprometer un calendario rígido en horas.

La gestión operativa del backlog se hace en Zube.io, una herramienta de gestión ágil integrada con GitHub Issues. El backlog completo (con sus épicas, sprints, estimaciones y estados) se mantiene en Zube, que actúa como fuente de referencia de la planificación; las historias se reflejan además como issues en el repositorio, de modo que la planificación y el código comparten un mismo hilo de trabajo y se evita la duplicación de información.

GitHub aloja el código y articula el flujo de trabajo. Cada historia se desarrolla en una rama de funcionalidad (feature branch) que se integra en la rama principal mediante un merge desde la rama de integración «develop». Cada integración constituye un punto explícito de revisión del incremento antes de incorporarlo, en línea con el espíritu de inspección continua de Scrum. Esta estructura facilita la trazabilidad: cualquier cambio en el código puede rastrearse, a través de su rama y del mensaje de commit, hasta la historia de usuario que lo motivó.

La entrega continua se apoya en GitHub Actions, que automatiza en cada pull request la ejecución del linter, la comprobación de tipos y la suite de pruebas con Vitest, y verifica además el build de producción de Next.js en cada integración sobre la rama principal. A esta verificación se suma un análisis estático continuo con SonarCloud, sincronizado con la rama principal del repositorio, que evalúa las métricas de seguridad, fiabilidad y mantenibilidad del código y exige mantener un Quality Gate verde como condición de integración. El despliegue al entorno de producción (reservas.iuce.usal.es) se ejecuta de forma manual al cierre de cada sprint, actualizando el código en el servidor y reconstruyendo y reiniciando los contenedores Docker mediante docker-compose, una decisión consciente para mantener bajo control el momento exacto en que un cambio llega al usuario final.

Adicionalmente, al cierre de los hitos relevantes del proyecto, el repositorio publica una release versionada en GitHub acompañada de un changelog detallado, capturas de la interfaz, evidencias de los tests y del análisis estático e instrucciones de despliegue local. La primera de estas publicaciones, v0.1.0, recoge el resultado de los cuatro primeros sprints; la v0.2.0 agrupa los sprints 5 a 7; y la v1.0.0 constituye la versión definitiva del sistema tras los sprints 8 y 9. Todas están disponibles en github.com/ShockyDEV/IUCE-Reservas-TFG/releases.

4.2. Alternativas estudiadas

Antes de fijar las tecnologías definitivas se evaluaron alternativas para los componentes con mayor impacto en el sistema: el framework web, la base de datos, el mecanismo de autenticación y el modelo de despliegue. La *tabla 1* resume las opciones consideradas y los criterios determinantes en cada caso. Los apartados siguientes desarrollan las dos decisiones cuya justificación requiere un comentario más extenso.

Tabla 1. Alternativas estudiadas para los componentes principales del sistema.

Componente	Elección final	Alternativas descartadas
Framework web	Next.js 14 + TypeScript	HTML/CSS/JS + Express, React + API REST
Base de datos	PostgreSQL 16 + Prisma	MySQL, MongoDB
Autenticación	Magic link (NextAuth + Resend)	SAML institucional, OAuth Google
Despliegue	Docker (app + PostgreSQL) tras Apache + Let's Encrypt	Node.js nativo con systemd, Vercel, Railway

4.3. Análisis comparativo del framework web: Next.js frente a HTML/CSS/JS clásico y a React con backend separado

La elección de Next.js como framework principal se tomó tras valorar dos alternativas habituales en el desarrollo web actual. La primera era construir el sistema con HTML, CSS y JavaScript clásicos en el navegador, sirviendo las páginas desde un backend mínimo en Node.js con Express. Esta vía es la más sencilla de explicar y la más cercana a lo visto en las prácticas del grado, pero renuncia a una serie de capacidades que Next.js trae resueltas de serie: enrutado basado en sistema de ficheros, recarga en caliente durante el desarrollo, empaquetado y división por código en producción y, sobre todo, una forma uniforme de transferir datos entre servidor y cliente con tipos compartidos. Para una aplicación con calendario, formularios validados y panel de administración, el coste de mantener todo ese andamiaje a mano resultaba desproporcionado. La segunda alternativa consistía en separar la aplicación en dos proyectos independientes: una SPA en React para la interfaz y una API REST en Node.js con Express para el servidor. Esta arquitectura es válida y ampliamente utilizada, pero supone duplicar configuración y herramientas de desarrollo, mantener dos despliegues paralelos y, sobre todo, sostener un contrato de API entre cliente y servidor cuya divergencia es una fuente constante de errores de integración. Next.js 14 con App Router cubre ambos lados con un único proyecto en TypeScript, renderiza por defecto en servidor mediante React Server Components y se integra de forma natural con Prisma y NextAuth, lo que se tradujo en menos tiempo dedicado a infraestructura y más a la lógica propia del problema de las reservas.

4.4. Autenticación: magic link frente a SSO institucional y OAuth con Google

La autenticación se valoró frente a tres opciones: integrar el sistema con la federación SAML (SSO) de la Universidad de Salamanca (idUSAL), delegar el acceso en cuentas de Google institucionales mediante OAuth, o implementar un esquema passwordless basado en magic link sobre el correo institucional del usuario. SAML habría sido la solución más limpia desde el punto de vista de identidad corporativa, pero requiere coordinación con el servicio de informática de la universidad, la habilitación de un Service Provider y un calendario de pruebas que no encajaba en los plazos del TFG. OAuth con Google funciona sin esa coordinación, pero deja fuera a cualquier usuario con dominio distinto de @usal.es y obliga a depender de un proveedor externo para validar identidades académicas. El magic link sobre correo institucional ofrece un compromiso razonable: aprovecha el directorio de correo que la universidad ya gestiona, no exige integración con servicios externos y reduce la superficie de ataque al no almacenar contraseñas en el propio sistema. La sustitución por SAML queda recogida como línea de trabajo futura para una eventual transferencia del sistema a otros institutos de la USAL.

4.5. Herramientas y tecnologías elegidas

El sistema se construye sobre un stack homogéneo en TypeScript, lo que permite emplear un único lenguaje de programación en todas las capas de la aplicación: desde la definición del esquema de datos y las consultas a la base de datos, pasando por la lógica de negocio en el servidor, hasta la interfaz que se ejecuta en el navegador. Esta unificación es posible gracias a dos piezas del stack. Por un lado, Next.js es un framework full-stack que ejecuta código TypeScript tanto en el servidor (Server Components y rutas de API) como en el cliente, eliminando la frontera de lenguaje que tradicionalmente separaba el frontend del backend. Por otro, Prisma actúa como capa de acceso a datos generando, a partir del esquema, un cliente TypeScript con tipos derivados directamente de las tablas, de modo que incluso las consultas a la base de datos se escriben y verifican en el mismo lenguaje.

La principal ventaja de este enfoque es que los tipos de datos se comparten de extremo a extremo: una entidad definida en el esquema se propaga, con el mismo tipo, hasta el componente de interfaz que la muestra, y el compilador detecta en tiempo de compilación cualquier incoherencia entre capas (por ejemplo, un campo renombrado en la base de datos que un formulario sigue esperando con el nombre antiguo). Esto reduce una clase entera de errores de integración que, en una arquitectura con lenguajes distintos en cada capa, solo se manifestarían en ejecución. Además, disminuye la carga cognitiva del desarrollo al no requerir el cambio constante de lenguaje y de herramientas entre el trabajo de servidor y el de cliente. Las versiones concretas de cada tecnología se fijaron al inicio del desarrollo y se han mantenido estables a lo largo de todos los sprints, salvo actualizaciones menores de seguridad.

El núcleo de la aplicación es Next.js 14 con App Router, ejecutado sobre Node.js 20 y escrito íntegramente en TypeScript 5. Next.js gestiona tanto el renderizado de la interfaz como las rutas de la API REST, evitando la necesidad de mantener un servidor separado. La capa de acceso a datos se apoya en Prisma 6 como ORM, que traduce el modelo definido en `schema.prisma` a las tablas reales de PostgreSQL 16 y proporciona un cliente de consultas tipado en tiempo de compilación. La autenticación se delega en NextAuth.js v5 (en su rama beta), configurado con un proveedor de credenciales propio que implementa un flujo de magic link: genera un token de un solo uso, lo persiste y envía el enlace de acceso a través de Resend, un servicio de correo transaccional con una API moderna.

La capa de presentación utiliza Tailwind CSS 3 como framework de estilos. La elección de Tailwind responde a tres criterios: permite trabajar directamente sobre el marcado sin saltar entre archivos de CSS, su sistema de tokens de diseño facilita mantener la coherencia visual entre componentes y su tamaño en producción es reducido gracias a la eliminación automática de clases no utilizadas. En la primera fase del desarrollo no se ha empleado una biblioteca de componentes preconstruidos, con el fin de conservar el control sobre la accesibilidad y el comportamiento responsive de cada pantalla; durante el Sprint 7 se introduce shadcn/ui como

base para acelerar el refactor visual sin sacrificar dicho control, ya que sus componentes se incorporan al proyecto como código propio y permanecen totalmente personalizables.

El aseguramiento de calidad del código se apoya en Vitest como framework de pruebas unitarias, con cobertura sobre las utilidades de validación y las máquinas de estado de la reserva, y en SonarCloud como plataforma de análisis estático continuo, integrada con el repositorio mediante su análisis automático y configurada para evaluar las métricas de seguridad, fiabilidad y mantenibilidad en cada push a la rama principal.

En producción, sobre el servidor solis.usal.es del CPD de la USAL, la aplicación Next.js (Node.js 20) y la base de datos PostgreSQL 16 se ejecutan como contenedores Docker orquestados mediante docker-compose. Apache 2 actúa como proxy inverso (HTTPS con certificado Let's Encrypt y WAF ModSecurity) hacia el contenedor de la aplicación (localhost:3000), y los datos persisten en un volumen Docker. En desarrollo, docker-compose se emplea para levantar PostgreSQL.

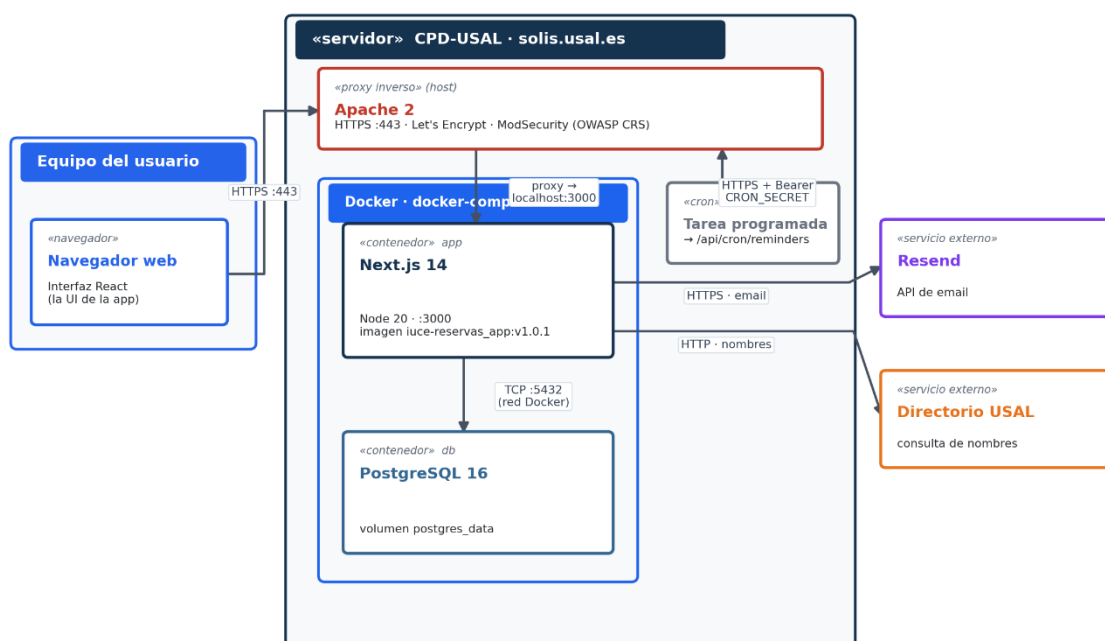


Figura 1. *Arquitectura de despliegue del sistema de producción.*

5. ASPECTOS RELEVANTES DEL DESARROLLO DEL PROYECTO

Este capítulo recoge las decisiones de diseño e implementación que han marcado el desarrollo de IUCE Reservas, agrupadas por el área del sistema sobre la que actúan. Para un recorrido exhaustivo por el código fuente existen el repositorio y la documentación técnica asociada; aquí se ofrece una justificación razonada de los caminos elegidos en aquellos puntos en los que existían alternativas no triviales. El orden de los apartados sigue, a grandes rasgos, el flujo natural del proyecto: primero el ciclo de vida que lo ha articulado, después las decisiones estructurales sobre el modelo de datos y los estados del dominio, a continuación los aspectos transversales (autenticación, concurrencia, arquitectura del frontend) y finalmente el aseguramiento de calidad que se ha mantenido en paralelo a todos ellos.

5.1. *Ciclo de vida y planificación adaptativa*

El proyecto se ha articulado siguiendo la metodología Scrum, adaptada al hecho de que se trata de un trabajo individual con un único desarrollador que asume simultáneamente los roles de Scrum Master, Product Owner y equipo de desarrollo. Las reuniones de revisión y planificación se han mantenido con los tutores actuando como Product Owners externos, lo que ha permitido conservar el espíritu del marco (retroalimentación frecuente, planificación a corto plazo, entrega incremental) sin forzar artificialmente una estructura de roles pensada para equipos.

Los sprints han tenido una duración aproximada de una semana. Las tutorías, celebradas cada dos semanas, han servido para revisar el avance acumulado de dos sprints y planificar los siguientes ajustes. Esta flexibilidad alinea la entrega con los hitos académicos reales, más que con un calendario teórico.

La gestión del Product Backlog se ha realizado en Zube, integrado con GitHub Issues. Cada historia de usuario es una tarjeta en Zube, con su épica, su columna en el kanban y su sprint asignado, y se refleja como issue en GitHub, vinculada al commit que la implementa. Esta organización proporciona dos vistas complementarias: una orientada al producto (kanban y seguimiento de sprints en Zube) y otra orientada al código (issues y commits en GitHub). El backlog gestionado en Zube ofrece así una traza ordenada desde la épica hasta el incremento que entrega cada funcionalidad.

Como complemento al ciclo iterativo se publican releases versionadas en GitHub al cierre de cada hito relevante. Cada release agrupa los sprints completados hasta la fecha y se acompaña de una nota detallada (changelog, capturas de interfaz, evidencias de tests y de análisis estático, instrucciones de despliegue local), quedando públicamente disponible como entregable autocontenido. La primera, v0.1.0, recoge los cuatro primeros sprints; la v0.2.0 incorpora los sprints 5 a 7; y la v1.0.0, que cierra el sprint final, constituye la versión definitiva

del sistema, lista para su despliegue en producción tras integrar los sprints 8 y 9. Más allá del entregable, preparar la nota de release funciona como punto de control: aflora documentación pendiente, tests que faltan y pasos de despliegue mal cubiertos.

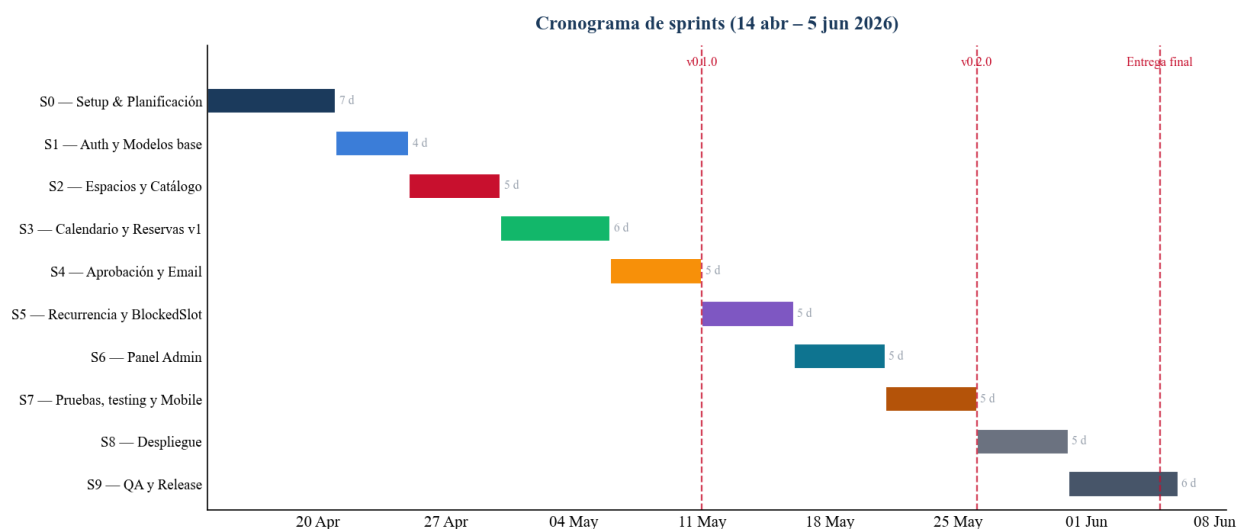


Figura 2. Cronograma de releases y sprints.

5.2. Modelo de datos: decisiones de diseño

El modelo relacional gira en torno a tres entidades nucleares (usuarios, espacios y reservas) y un conjunto auxiliar que cubre la autenticación, los bloqueos administrativos y la auditoría. La decisión interesante en esta área pasa por cómo se han modelado las relaciones y, sobre todo, cómo se representa la reserva recurrente. Qué entidades existen viene dado por el dominio y no requiere debate. Conviene señalar una decisión deliberada de diseño: las notificaciones que ve el usuario no se almacenan como entidad propia, sino que se derivan en tiempo de consulta a partir del estado de las reservas (por ejemplo, las solicitudes pendientes de aprobación para un administrador, o las reservas del usuario revisadas recientemente), evitando así una tabla y la lógica de mantenimiento de su ciclo de vida.

Los modelos de NextAuth (Account, Session y VerificationToken) se mantienen tal cual los define el esquema oficial. Se valoró fusionar VerificationToken con la entidad User, pero conservar la separación tiene dos ventajas claras: permite actualizar la librería de autenticación sin riesgo de divergencia en el esquema y deja la puerta abierta a otros proveedores de identidad (SSO institucional, OAuth con Google) reutilizando la misma tabla. El coste es despreciable: tres tablas pequeñas, relaciones bien delimitadas, obtenemos flexibilidad.

La entidad Reservation almacena cada reserva como un registro independiente con fecha y hora de inicio y fin, espacio asociado, usuario solicitante y estado actual. Se descartó la alternativa de seguir el patrón RFC 5545 (un único registro con reglas de recurrencia RRULE interpretadas en tiempo de consulta) por dos motivos. Primero, complejidad operativa: cancelar o modificar una instancia concreta de una serie obliga a introducir excepciones (EXDATE), una fuente conocida de bugs en sistemas de calendarios. Segundo, complejidad de consulta: detectar solapamientos requiere expandir las reglas de recurrencia para cada espacio, lo que estorba al uso de índices y mete lógica de calendarios en cada consulta de disponibilidad. Almacenar cada instancia como un registro independiente convierte el solapamiento en una consulta SQL sobre un índice compuesto (espacio, inicio, fin), a cambio de generar N filas al crear una serie de N instancias: un coste asumible para los volúmenes esperados en el IUCE.

BlockedSlot se ha separado a propósito de Reservation pese a representar también una franja horaria ocupada en un espacio. La razón es semántica: un bloqueo administrativo no tiene solicitante, no transita por un flujo de aprobación y no genera notificaciones; tiene origen, motivo y duración, y nada más. Fusionarlo con Reservation habría exigido permitir nulos en relaciones clave (usuario, estado) y contaminar la máquina de estados con casos especiales. Manteniéndolo aparte, el código que comprueba disponibilidad simplemente busca solapamientos en ambas tablas con la misma forma de consulta y el panel administrativo gestiona bloqueos sin pasar por la lógica de reservas.

AuditLog es estructuralmente sencillo, pero cumple un papel importante: registra de forma inmutable las acciones críticas del sistema (aprobaciones, rechazos, cancelaciones administrativas, creación de bloqueos) con su autor y su instante. No interviene en la lógica funcional, pero da la trazabilidad necesaria para justificar ante la dirección del IUCE cualquier decisión tomada por un administrador sobre una reserva ajena y evitar problemas con los usuarios finales que harán uso del producto.

5.3. Máquina de estados de la reserva

Una reserva no es un objeto estático: nace como solicitud, pasa por una decisión administrativa y termina confirmada, rechazada, cancelada o caducada. Modelar ese ciclo de vida con un conjunto cerrado de estados explícitos, en lugar de un puñado de campos booleanos sueltos, es de las decisiones que más ha simplificado la capa de servicio.

El tipo enumerado ReservationStatus define cinco estados: PENDING (solicitud creada, a la espera de aprobación), APPROVED (aprobada y confirmada para el usuario), REJECTED (rechazada con motivo registrado), CANCELLED (cancelada antes de la fecha, ya sea por el propio solicitante o de oficio por un administrador) y EXPIRED (reserva previamente aprobada cuya hora de fin ya ha pasado; la marca de oficio una tarea programada). El estado EXPIRED se aplica únicamente a reservas que estaban en estado APPROVED y ya han finalizado, no a solicitudes pendientes. Las transiciones forman un conjunto cerrado: desde PENDING se pasa

a APPROVED, REJECTED o CANCELLED; desde APPROVED se pasa a CANCELLED (cancelación por el usuario o un administrador), a REJECTED (anulación administrativa de una reserva ya aprobada) o a EXPIRED (caducación automática). La anulación administrativa de una reserva APPROVED la deja en estado REJECTED y es indistinguible para el usuario de un rechazo en revisión; la distinción se preserva únicamente en el registro de auditoría (acción RESERVATION_REJECTED_AFTER_APPROVAL) y en un correo de anulación específico. No se distingue a nivel de modelo la cancelación del usuario de la del administrador: ambas comparten el estado CANCELLED, y la autoría queda registrada por separado en el registro de auditoría.

Se evaluó incorporar una librería de máquinas de estado finitas (XState o similar). Se descartó: el coste de añadir la dependencia, su modelo mental y su superficie de configuración era desproporcionado para cinco estados y un puñado de transiciones. La transición central del flujo, la decisión administrativa desde PENDING hacia APPROVED o REJECTED, se gobierna mediante una función pura que valida si la transición es legal para el rol del solicitante y devuelve un booleano; cuando la transición no está permitida, el handler responde con un error 409; al estar aislada en una única función, es sencilla de probar y evita repetir la regla en cada controlador. Las restantes transiciones se resuelven en sus propios puntos de entrada por su distinta naturaleza: la cancelación se valida en el endpoint que la atiende, y la caducación a EXPIRED la aplica una tarea programada (cron) que recorre las reservas cuya hora de fin ya ha pasado. Asimismo, la anulación administrativa dispone de su propio punto de entrada: el endpoint PATCH de administración discrimina por el estado actual de la reserva, de modo que si está APPROVED y se rechaza, ejecuta el flujo de anulación (handleAdminAnul), con validación propia (exige status = REJECTED), acción de auditoría específica (RESERVATION_REJECTED_AFTER_APPROVAL) y un correo de anulación distinto.

La mayoría de las transiciones explícitas (aprobación, rechazo, anulación, modificación y cancelación) disparan un correo al usuario implicado con el detalle del cambio; la caducación automática a EXPIRED se aplica de forma masiva por el cron y no envía notificación. El envío se ejecuta tras confirmar la escritura en base de datos y es best-effort: si falla, se registra en el log (console.error) y se descarta, sin revertir el estado de la reserva y sin reintentar el envío.

El administrador puede, además de aprobar o rechazar solicitudes pendientes: (a) modificar los detalles de una reserva en estado PENDING o APPROVED (título, descripción, fecha y horas, asistentes y notas), validándose de nuevo el solapamiento y notificando al usuario con un correo que muestra tachados los datos anteriores; y (b) anular una reserva ya aprobada, que pasa a estado REJECTED, con una notificación específica al usuario. Ambas acciones quedan registradas en el log de auditoría (RESERVATION_MODIFIED_BY_ADMIN y RESERVATION_REJECTED_AFTER_APPROVAL, respectivamente).

5.4. Autenticación passwordless con NextAuth

La autenticación se resuelve con NextAuth.js v5 mediante un proveedor de credenciales propio que implementa el flujo de magic link sobre el servicio transaccional Resend. La justificación teórica del modelo passwordless está en el capítulo 3; aquí interesa qué ha supuesto la decisión a nivel de implementación.

El flujo es el habitual: el usuario introduce su correo institucional, el sistema genera un token de un solo uso con caducidad corta, lo almacena en VerificationToken y envía a su correo un enlace con un token aleatorio de un solo uso. Al hacer clic, el sistema valida el token contra VerificationToken, crea la sesión mediante NextAuth y redirige al dashboard. La ausencia de contraseñas elimina un vector de ataque importante y libera al desarrollo de toda la lógica asociada (almacenamiento seguro de hashes, políticas de complejidad, recuperación, rotación), pero introduce dos dependencias críticas: el correo tiene que llegar de forma fiable y el dominio remitente tiene que estar correctamente autenticado (SPF, DKIM, DMARC) para no caer en spam. Resend cubre la parte transaccional; la verificación de dominio se ha integrado en el procedimiento de despliegue.

Sobre este flujo se han añadido tres piezas propias. Primero, una capa de roles (USER, ADMIN, SUPER_ADMIN) almacenada en la entidad User y leída en el callback de sesión, de forma que cada petición autenticada conoce el rol sin un viaje extra a base de datos. Segundo, la protección de rutas se realiza en los layouts de servidor de los grupos privados, que redirigen a la pantalla de acceso cuando no existe sesión, y en cada handler de la API, que verifica la sesión y el rol antes de procesar la petición. Y tercero, un mock de autenticación activable por variable de entorno (NEXT_PUBLIC_USE MOCK_AUTH=true) que permite trabajar en local sin enviar correos reales y sin depender del servicio externo. Esta última pieza ha resultado especialmente útil para los tests automatizados y para los entornos de revisión, donde activar el proveedor real introduciría latencia y consumo innecesario de cuota, así como para el testeo de la aplicación durante el seguimiento tutorizado. El modelo User incluye además la preferencia receiveAdminEmails (booleano, activado por defecto): cada administrador puede silenciar los avisos por correo de nuevas solicitudes sin perder su rol. Esta preferencia solo pueden modificarla los usuarios con rol ADMIN o SUPER_ADMIN, desde su perfil (un usuario sin esos roles recibe un error 403), y se renderiza como un interruptor en la página de perfil.

5.5. Validación de solapamientos y concurrencia

El problema funcional más delicado del sistema es impedir que dos reservas confirmadas se solapen sobre el mismo espacio. Detectar el solapamiento no es complejo. El razonamiento sobre relaciones entre intervalos temporales fue formalizado por Allen (1983), cuyo álgebra distingue trece relaciones posibles entre dos intervalos. La condición que interesa aquí (que dos

reservas compartan algún instante, con independencia de cuál sea la relación exacta entre ellas) abarca varias de esas relaciones y se operacionaliza mediante una única comparación entre los extremos de ambas franjas: dos intervalos se intersecan cuando cada uno comienza antes de que el otro termine, lo que en la consulta se expresa como:

$$\text{fin_existente} > \text{inicio_nuevo} \quad \text{AND} \quad \text{inicio_existente} < \text{fin_nuevo}$$

La dificultad no está, por tanto, en la consulta, sino en garantizar que dos reservas confirmadas no acaben solapadas. La creación de una solicitud comprueba primero la disponibilidad de la franja frente a las reservas existentes en estado PENDING o APPROVED y, si está libre, la registra. Estas dos operaciones no se ejecutan dentro de una misma transacción, de modo que, en teoría, dos solicitudes para la misma franja que llegasen casi a la vez podrían ambas encontrar la franja libre y crearse. Ahora bien, en este sistema ese escenario no produce un solapamiento real de reservas confirmadas, porque la confirmación nunca es automática: toda reserva nace en estado PENDING y requiere una aprobación administrativa explícita.

La salvaguarda efectiva frente al doble solapamiento se sitúa, por tanto, en el momento de la aprobación. Cuando un administrador aprueba una solicitud PENDING, el endpoint de aprobación vuelve a comprobar la disponibilidad de la franja, esta vez frente a las reservas ya APPROVED. Si en el intervalo entre la creación de la solicitud y su aprobación se hubiera confirmado otra reserva conflictiva, esta segunda comprobación lo detecta y bloquea la aprobación. De este modo, aunque puedan coexistir varias solicitudes PENDING solapadas, el sistema nunca llega a confirmar dos reservas que se pisen sobre el mismo espacio. La misma validación de disponibilidad se ejecuta además en el cliente antes de enviar la solicitud, con un propósito distinto: dar retroalimentación inmediata al usuario y reducir las peticiones que llegan al servidor solo para fallar; la verdad sobre la disponibilidad, no obstante, sigue estando en el servidor.

Esta solución es proporcionada a la escala de uso esperada en el IUCE, donde el volumen de solicitudes simultáneas sobre una misma franja es muy bajo y el flujo de aprobación manual actúa como punto de control natural. En un escenario de mayor concurrencia, la línea de evolución evidente sería envolver la comprobación de disponibilidad y la creación de la reserva en una única transacción con un nivel de aislamiento estricto, de forma que la propia base de datos serialice los accesos concurrentes a las mismas filas y se impida incluso la creación de solicitudes PENDING solapadas. Se ha optado por no incorporar ese mecanismo en esta versión por no estar justificado para la carga real del sistema, dejándolo señalado como posible mejora futura.

5.6. Arquitectura del frontend: App Router y Server Components

La aplicación está construida sobre el App Router de Next.js 14, lo que implica Server Components por defecto y la directiva `use client` solo en los puntos que requieren interactividad real (formularios con validación en vivo, calendarios, modales). La decisión viene heredada al elegir el framework, pero tiene consecuencias prácticas que conviene mencionar.

La principal es que la mayoría de las páginas (catálogo de espacios, lista de reservas del usuario, panel de aprobación) se renderizan en servidor con acceso directo al cliente de Prisma, sin exponer un endpoint REST intermedio ni gestionar estados de carga en cliente. Lo que llega al navegador es el HTML resultante y el JavaScript justo para las islas interactivas, lo que mejora tanto el rendimiento percibido como la accesibilidad básica: el contenido viene en el HTML inicial, no se construye dinámicamente.

Las mutaciones (crear reserva, aprobar, rechazar, cancelar) se han implementado como rutas de API REST (Route Handlers del App Router, bajo `/api/`), invocadas desde el cliente mediante `fetch`. Cada operación expone un endpoint con su método HTTP (POST para crear, PATCH para aprobar, rechazar o cancelar), recibe y devuelve JSON, y valida la entrada en servidor con esquemas Zod antes de tocar la base de datos. La capa de cliente envía la petición con `fetch`, gestiona el estado de carga y procesa la respuesta y los códigos de estado tipados que devuelve la ruta. Este enfoque mantiene un contrato de transporte explícito y uniforme para todas las mutaciones, lo que facilita su prueba aislada y su consumo desde cualquier cliente. Además, un administrador puede modificar una reserva en estado PENDING o APPROVED mediante `PUT /api/admin/reservations/[id]` (título, descripción, fecha y horas, asistentes y notas), validado por el esquema Zod `adminUpdateReservationSchema`, y anular una reserva ya aprobada mediante PATCH con destino REJECTED.

Toda la superficie de mutación y consulta se sirve, por tanto, a través de rutas de API REST. Algunas son superficies estables pensadas para que un día pueda consumirlas un tercero, como el flujo de NextAuth y la API pública del catálogo de espacios; otras son internas de la aplicación, como la creación o revisión de reservas y los bloqueos administrativos. Mantener un único estilo de contrato (REST + JSON validado con Zod) para todas ellas aporta uniformidad y simplifica las pruebas, sin renunciar a la flexibilidad cuando hace falta. A esta superficie se suman dos endpoints de consulta que sustentan el calendario de disponibilidad: `GET /api/spaces/availability`, que devuelve los identificadores de los espacios ocupados en una franja dada (requiere sesión), y `GET /api/spaces/[id]/occupancy`, que devuelve los intervalos ocupados de un espacio en una ventana temporal (con tipo `RESERVATION_APPROVED`, `RESERVATION_PENDING` o `BLOCKED`).

5.7. Aseguramiento de calidad

El aseguramiento de calidad se ha planteado como práctica continua, no como una fase final del proyecto. Durante el ciclo se han mantenido dos líneas de defensa que se ejecutan automáticamente en cada cambio enviado a la rama principal: las pruebas unitarias con Vitest y el análisis estático con SonarCloud. En la fase final del desarrollo se incorporó una tercera línea, el análisis de estilo y errores mediante un linter (ESLint con la configuración recomendada de Next.js y reglas de TypeScript estricto), que consolida el aseguramiento de calidad al detectar problemas de estilo y patrones propensos a error antes incluso de la ejecución de las pruebas.

Los tests de Vitest se han concentrado a propósito en los puntos donde un fallo silencioso tiene consecuencias funcionales reales: validación de fechas y franjas, tabla de transiciones de la máquina de estados, utilidades de cálculo de solapamientos y serializadores de las rutas de API. Hemos buscado una cobertura intensa sobre la lógica de negocio antes que una cobertura uniforme sobre todo el código, ya que los tests de componentes triviales aportan poca señal. Vitest se ha elegido sobre Jest por su integración nativa con Vite y por reaprovechar directamente la configuración de TypeScript del proyecto, ahorrando la duplicidad de configuración que Jest exige.

SonarCloud se integra mediante su análisis automático vinculado al repositorio y se ejecuta en cada push a main y en cada pull request abierto contra esa rama. El Quality Gate se ha mantenido en su configuración por defecto (cobertura sobre nuevo código, sin bugs ni vulnerabilidades, deuda técnica acotada, cero issues de seguridad) y se ha conservado verde durante todo el ciclo de desarrollo. Su valor en este proyecto no está tanto en cazar errores graves (de eso ya se ocupan el linter y los tipos de TypeScript) como en mantener visibilidad sobre la deuda técnica acumulada y obligar a pequeñas refactorizaciones que, sin esa presión, se aplazarían indefinidamente.

Por último, el pipeline de GitHub Actions verifica que el build de producción se completa sin errores en cada integración sobre main. Aparentemente trivial, este control ha resultado más útil de lo previsto: caza pronto los errores de tipos que solo afloran en compilación, las referencias rotas tras un renombrado y las dependencias eliminadas del package.json sin retirar sus importaciones del código.

5.8. Panel de métricas y analítica de uso

Más allá del flujo operativo de solicitud y aprobación, el sistema incorpora un panel de métricas accesible únicamente al rol administrador, que agrega la información de las reservas registradas para ofrecer una visión de conjunto del uso de los espacios. La motivación de esta funcionalidad es convertir los datos que el sistema ya genera de forma natural durante su

operación en información útil para la toma de decisiones del instituto, sin requerir ninguna herramienta externa ni exportación manual de datos. Con independencia del panel de métricas, el sistema ofrece una exportación administrativa de reservas: `GET /api/admin/exports/reservations-calendar?month=YYYY-MM` descarga un fichero CSV mensual (codificación UTF-8 con BOM y separador ";", apto para Excel en español) con las reservas APPROVED y PENDING del mes (columnas: Título, Espacio, Código, Usuario, Email, Fecha, horas, Asistentes, Estado, Revisado por y Notas), registrando la acción en el log de auditoría (EXPORT_CSV). El formato es CSV.

El panel presenta indicadores agregados de uso a partir de las reservas almacenadas, de forma que el personal autorizado puede valorar la demanda real de cada espacio y detectar patrones de ocupación. Al construirse sobre los mismos datos que sustentan el flujo de reservas, las cifras mostradas son siempre coherentes con el estado real del sistema y no dependen de procesos de sincronización adicionales. La consulta se realiza en servidor mediante agregaciones sobre la base de datos, aprovechando los índices descritos en la sección 5.2 (*véase p. 17, sección 5.2.*), lo que mantiene el coste de generar el panel acotado incluso conforme crece el volumen histórico de reservas.

Desde el punto de vista del diseño, este panel ilustra un principio recurrente en el proyecto: extraer el máximo valor de los datos que el sistema ya posee antes de incorporar fuentes o componentes nuevos. La misma información que da soporte a la operación diaria, debidamente agregada, sostiene una segunda capa analítica que amplía el alcance del sistema desde la mera gestión de reservas hacia el apoyo a la planificación de los espacios del instituto.

5.9. Rendimiento y consumo de recursos

Aunque IUCE Reservas no es un sistema de alta concurrencia (el volumen de usuarios esperado se mide en decenas simultáneas), conviene caracterizar su comportamiento en términos de rendimiento y consumo de recursos para validar que el dimensionamiento del despliegue es razonable y para detectar puntos en los que un cambio de carga futuro podría tensar el sistema.

En tiempos de respuesta, las páginas renderizadas como Server Components (catálogo de espacios, dashboard del usuario, panel de aprobación) se sirven con latencias dominadas por la consulta a base de datos y el render en servidor. Las consultas más exigentes son las de detección de solapamientos sobre la tabla Reservation. Desde el punto de vista de complejidad algorítmica, sin índice, esta operación tendría un coste lineal $O(n)$ sobre el número de reservas del espacio, que para los volúmenes anuales del IUCE bastaría pero no sería lo ideal. El índice compuesto sobre (espacio, inicio, fin) descrito en la sección 5.2 lo reduce a $O(\log n)$ gracias a la estructura B-tree con la que PostgreSQL materializa los índices por defecto, una organización descrita en detalle por Comer (1979). Cormen et al. (2009) demuestran que "every n -node B-tree has height $O(\lg n)$ " y, en consecuencia, que los B-trees "can also be used to implement

many dynamic-set operations in time $O(\lg n)$ ", lo que cubre las operaciones de búsqueda, inserción y borrado que ejecuta Prisma sobre el índice. El resto de las operaciones de lectura se apoyan en los índices declarados explícitamente en el esquema de Prisma (anotaciones @@index), definidos en su mayoría sobre las columnas de clave foránea, con el mismo coste logarítmico.

El consumo de memoria del contenedor de la aplicación se mantiene estable en valores compatibles con un contenedor de tamaño reducido, ya que el modelo de Server Components evita acumular estado de sesión en cliente y las conexiones a PostgreSQL se gestionan mediante un pool acotado configurado en el cliente de Prisma. No se ha detectado fuga de memoria en periodos prolongados de ejecución; las pequeñas oscilaciones observadas se corresponden con el ciclo natural del recolector de basura de V8.

El tamaño del bundle JavaScript que llega al cliente es modesto gracias a la división por defecto de Next.js en chunks por ruta: cada página carga únicamente el código asociado a sus componentes interactivos y comparte un runtime común cacheado por el navegador. Las dependencias pesadas (cliente de Prisma, lógica de email, configuración de NextAuth) viven en servidor y no se envían al navegador.

Respecto a la base de datos, el volumen de datos esperado para la operación anual del IUCE (orden de miles de reservas, no millones) está holgadamente por debajo de los umbrales en los que PostgreSQL requiere afinado específico. La estrategia de almacenar cada instancia recurrente como un registro independiente (*véase p. 17, sección 5.2.*) eleva el número de filas frente a un modelo basado en reglas RRULE, pero el crecimiento sigue siendo lineal con el uso real y, gracias a los índices descritos, las consultas críticas conservan el coste logarítmico que Cormen et al. (2009) acreditan para las operaciones sobre B-trees, con independencia del tamaño acumulado de la tabla.

El cuello de botella práctico del sistema vive fuera de la aplicación y de la base de datos: lo aporta el servicio externo de envío de correo electrónico, cuyo tiempo de entrega de un magic link o de una notificación de aprobación depende íntegramente de la latencia de Resend y de la cola del proveedor de correo del destinatario. Para amortiguar este coste, los envíos se ejecutan de forma asíncrona tras la confirmación en base de datos (*véase p.18, sección 5.3.*) y no bloquean la respuesta al usuario.

5.10. Ingeniería de software como marco del proceso

Las decisiones técnicas descritas a lo largo de este capítulo se han tomado dentro de un marco más amplio de ingeniería del software que ha guiado el ciclo completo: análisis de requisitos, modelado del dominio, diseño arquitectónico, implementación incremental, verificación y documentación. Sommerville (2016) describe este conjunto de actividades como un proceso disciplinado que distingue al desarrollo profesional de la mera escritura de código

y que se aplica con independencia de la metodología concreta (ágil o predictiva) que articule las iteraciones.

La construcción del modelo de datos, en concreto, ha seguido el procedimiento canónico de la disciplina. Se partió del análisis del dominio (espacios reservables, usuarios con distintos roles, ciclo de vida de las reservas, bloqueos administrativos) para identificar las entidades nucleares y sus relaciones, expresadas inicialmente como un diagrama entidad-relación. Sobre esa primera abstracción se aplicaron las reglas de normalización del modelo relacional descritas por Date (2003) hasta llegar a un esquema en tercera forma normal, libre de dependencias transitivas y redundancias controlables. El esquema resultante se materializó como un fichero `schema.prisma` que actúa como fuente única de verdad: a partir de él se genera el cliente TypeScript tipado que usan las rutas de API y las páginas renderizadas en servidor, y se sincroniza directamente el esquema con PostgreSQL (flujo `prisma db push`), sin migraciones versionadas. La trazabilidad entre el dominio conceptual y la implementación, por tanto, es directa y revisable: cualquier cambio en una entidad nace en este fichero y se propaga al resto del sistema sin pasos manuales intermedios.

Como complemento al código, el proyecto produce un conjunto de manuales que documentan cada vertiente del sistema y que se entregan junto a esta memoria como anexos: un manual de arquitectura que describe la disposición de capas, el flujo de datos entre frontend y base de datos y las decisiones de despliegue; un manual de API que enumera los endpoints REST, tanto los públicos como los internos de la aplicación, con su contrato de entrada/salida y los códigos de error previstos; un manual de usuario orientado al personal del IUCE, con capturas del flujo completo de solicitud y consulta de reservas; y un manual de administración dirigido al personal autorizado, que cubre la aprobación de solicitudes, la gestión de espacios y bloqueos, y la consulta del registro de auditoría. Como recoge Sommerville (2016), la separación de la documentación por audiencia es una práctica estándar en ingeniería del software y permite que cada lector encuentre exclusivamente la información relevante para su rol, sin tener que filtrar contenido pensado para otros perfiles.

5.11. El papel de la IA generativa en el proyecto

La irrupción acelerada de la inteligencia artificial generativa en los últimos años ha modificado las condiciones materiales del trabajo intelectual y, en particular, del desarrollo de software. Lo que hace apenas un lustro era objeto de investigación especializada (grandes modelos de lenguaje, modelos de difusión para imagen, modelos multimodales) hoy se incorpora con normalidad en herramientas que cualquier desarrollador utiliza a diario, hasta el punto de que ignorar deliberadamente su existencia en un proyecto técnico actual sería una decisión metodológica difícil de sostener. La cuestión relevante hoy es cómo integrar estas herramientas sin que su uso comprometa la autoría intelectual del trabajo ni desplace el criterio profesional que lo articula; debatir si conviene usarlas o no es ya una pregunta superada.

El debate académico reciente ha desplazado el foco desde la mera capacidad técnica de los modelos hacia los marcos de uso responsable que deben acompañarlos. García-Peñalvo (2025) propone una vía proporcionada que gradúa la relación entre el profesional y la IA en escenarios de autonomía creciente (apoyo responsable, colaboración guiada y cocreación con declaración reforzada), bajo el principio común de la IA como complemento sometido a juicio humano y nunca como sustituto. Gerlich (2025), desde el ángulo cognitivo, alerta del riesgo simétrico: cuando la dependencia de estas herramientas crece sin contrapeso reflexivo, las funciones de pensamiento crítico se externalizan y la calidad del trabajo termina dictada por la herramienta, no por quien la opera. En esta misma línea, Armenteros Cosme et al. (2026) sistematizan el uso crítico, responsable y aplicado de la IA generativa, marco formativo en el que se apoya el enfoque adoptado en este proyecto.

La herramienta de IA generativa empleada en el proyecto fue Claude, de Anthropic. Las tareas concretas para las que se utilizó han sido cinco. Primero, la generación de los diagramas que acompañan al modelo de datos y al ciclo de vida de la reserva. Es importante subrayar que el modelo subyacente no es producto de la IA: la estructura entidad-relación, las decisiones de normalización, la separación entre Reservation y BlockedSlot, y el conjunto cerrado de estados de la reserva son fruto del análisis del dominio descrito en los apartados anteriores y se materializaron primero en el fichero `schema.prisma` del repositorio. La IA actuó únicamente como herramienta de visualización: a partir del esquema definido por el autor, generó los diagramas que aparecen en este documento y en los anexos.

Segundo, revisiones de código en el rol de revisor adicional para tareas concretas (detección de inconsistencias de tipado en TypeScript, sugerencias de nombres más claros para funciones, identificación de duplicación susceptible de refactor). Toda sugerencia se evaluó manualmente y se aceptó, modificó o rechazó según el criterio del autor.

Tercero, asistencia en la redacción de los manuales orientados al usuario final y al administrador, donde el reto principal es comunicar con claridad procedimientos técnicos a audiencias no técnicas. En todos los casos, la IA generó borradores que el autor revisó y reescribió antes de incorporarlos a la documentación final.

Cuarto, la asistencia en la escritura de las pruebas unitarias, que permitió acelerar la creación del conjunto de la sección de pruebas. Como en los casos anteriores, cada prueba generada se revisó y validó contra el comportamiento esperado del sistema antes de incorporarla.

Quinto, el brainstorming de ideas para la interfaz de usuario (UI) de la aplicación, entre las que se eligió después la propuesta que mejor se ajustaba a los requisitos y a las indicaciones de diseño del proyecto.

Estas interacciones se han apoyado en tres principios. El primero es que la dirección técnica y las decisiones de diseño permanecen en el lado humano. Como argumenta García-Peñalvo (2025), la relación entre el profesional y la IA generativa en contextos productivos debe evolucionar desde el apoyo pasivo hacia un modelo de cocreación en el que el criterio

profesional se amplifica a través de la tecnología en lugar de delegarse en ella. En este proyecto, ninguna decisión estructural (qué entidades modelar, cómo resolver la concurrencia en la creación de reservas, qué framework adoptar) se tomó por delegación; todas son fruto del análisis y la justificación recogidos en los apartados previos. El segundo principio es la verificación obligatoria de cualquier salida generada. Las revisiones de código se contrastaron contra la comprobación de tests; los diagramas, contra el esquema real de Prisma; los borradores de manual, contra el comportamiento real del sistema. La tercera consigna es la documentación honesta del uso, que es justamente el propósito de este apartado.

Conviene también señalar lo que la IA no es capaz de articular por sí misma. No identifiqué por sí sola los requisitos funcionales del IUCE, que se alcanzaron en conversaciones con la dirección del instituto y con los tutores del TFG. No propuso el modelo de roles (USER, ADMIN, SUPER_ADMIN) ni la separación entre solicitud, aprobación y cancelación, decisiones que nacen del análisis del flujo administrativo real. Tampoco eligió las tecnologías del proyecto, decisión tomada tras la evaluación comparativa descrita en el *Capítulo 4*. En este sentido, estamos en línea con las palabras de García-Peñalvo en el que la “IA acelera la materialización de ideas, no su concepción”. Coincide aquí la observación de Gerlich (2025) sobre el riesgo de la descarga cognitiva: trasladar las decisiones de criterio a la herramienta produce un trabajo sin autoría intelectual real, por pulido que parezca el resultado. Dominar estas herramientas exige, en consecuencia, pensar con claridad antes de escribir el prompt, mucho más que perfeccionar el prompt en sí.

Para enmarcar lo anterior conviene una última precisión. La IA generativa, tal y como la describen García-Peñalvo y Vázquez-Ingelmo (2023) en su revisión sistemática del campo, es una familia de técnicas (grandes modelos de lenguaje, modelos de difusión para imagen, modelos multimodales) capaces de producir contenido nuevo a partir de patrones aprendidos sobre datos masivos. Su utilidad en el desarrollo de software actual es real, pero conviene recordar que se trata de un amplificador, no de un sustituto del criterio profesional. La diferencia entre un proyecto bien dirigido y uno deficiente, en presencia de estas herramientas, sigue residiendo donde siempre ha residido: en la calidad del análisis, en la disciplina del proceso y en la honestidad con la que se documenta el trabajo realizado.

6. TRABAJOS RELACIONADOS

Antes de optar por un desarrollo a medida conviene situar IUCE Reservas frente a las soluciones ya existentes para la gestión de reservas de espacios. El análisis se centra en las categorías de herramientas de código abierto consolidadas. De esa comparación se desprende la justificación de la decisión adoptada.

En el ámbito del código abierto destacan dos referentes. MRBS (Meeting Room Booking System), iniciado por Gardner (s.f.), es una aplicación web libre bajo licencia GPL, escrita en PHP sobre MySQL o PostgreSQL, que organiza los espacios en áreas y salas, ofrece una vista de calendario en paralelo, admite reservas recurrentes y soporta múltiples idiomas, incluido el español, con autenticación configurable mediante mecanismos como LDAP.

Booked Scheduler (Korbel, 2020), antes phpScheduleIt, también libre y en PHP, es la más interesante en prestaciones de administración: cuotas de uso, aprobación de reservas, exportación a iCalendar, recordatorios e integración con LDAP y Active Directory. Ambas son fiables y están ampliamente desplegadas, pero comparten dos limitaciones para este proyecto. La primera es tecnológica: responden al modelo de aplicación PHP tradicional con renderizado íntegro en servidor sobre bases de código de larga trayectoria, alejadas del stack moderno que se quería emplear como ejercicio formativo. La segunda es funcional: su catálogo de funcionalidades excede ampliamente lo que el IUCE necesita, y adaptarlas a su identidad y a su flujo de aprobación exigiría intervenir sobre código heredado de terceros. A ello se añade que la última versión de código abierto de Booked Scheduler data de 2020 y su continuidad depende hoy del fork comunitario LibreBooking (s.f.), lo que introduce incertidumbre sobre el soporte a largo plazo de la herramienta.

En conjunto, ninguna de las dos categorías resolvía el problema del IUCE mejor que una aplicación propia. Las soluciones libres aportan madurez a costa de un exceso de funcionalidades, y las comerciales evitan el mantenimiento a costa de un coste recurrente y una dependencia externa. Desarrollar una aplicación a medida, ajustada al flujo del IUCE y construida sobre tecnologías actuales, resulta la opción más alineada con los objetivos formativos del trabajo y produce, además, un sistema más sencillo de entender, mantener y evolucionar por quien lo recibe.

7. CONCLUSIONES Y LÍNEAS DE TRABAJO FUTURAS

7.1. Conclusiones

La conclusión principal de este Trabajo de Fin de Grado es que la aplicación de una disciplina de ingeniería del software rigurosa, sostenida a lo largo de todo el ciclo, ha producido un sistema con calidad de producto. IUCE Reservas resuelve de extremo a extremo el problema que se planteó (autenticación institucional, motor de reservas con control de concurrencia, flujo de aprobación, notificaciones, panel administrativo, panel de métricas y despliegue en producción) y lo hace sobre una base mantenible, documentada y trazable. Si este desarrollo se hubiera realizado en un contexto empresarial en lugar de académico, el resultado sería directamente explotable: un producto entregable al cliente (el propio IUCE, cuyo personal de administración y servicios es su usuario final) y mantenible por un equipo de desarrollo distinto del que lo construyó, gracias a la documentación técnica y a la trazabilidad del código.

Esa calidad de producto es consecuencia directa de la disciplina del proceso. La adaptación de los métodos ágiles descrita en la sección 3.5. (*véase p. 9*) (sprints de duración acotada, un backlog vivo y la revisión periódica con la dirección) impuso un ritmo que evitó tanto la parálisis del análisis interminable como la deriva del desarrollo sin control. Sobre esa cadencia se apoyaron las prácticas que distinguen un sistema mantenible de un amasijo de código que funciona por casualidad: la trazabilidad entre historia de usuario, issue y commit; el esquema de datos como la raíz de la aplicación desarrollada; un conjunto de herramientas de pruebas concentrada en la lógica de negocio; y un Quality Gate que no deja crecer la deuda técnica sin control. La ingeniería del software entendida como proceso (análisis del problema, modelado, diseño, verificación continua y documentación) es, en última instancia, lo que ha convertido un encargo de varios meses en un producto que el IUCE podrá mantener más allá de la entrega académica.

La segunda conclusión atañe al papel que la inteligencia artificial generativa ha desempeñado en alcanzar ese resultado. García-Peñalvo (2025) sitúa la relación madura entre el profesional y estas herramientas en un modelo de cocreación donde el criterio humano no se sustituye, se amplifica. Este proyecto es una materialización concreta de ese principio: las decisiones que han determinado la calidad del sistema, descritas a lo largo del capítulo 5, fueron decisiones de criterio tomadas por el autor tras analizar el problema, no respuestas dictadas por la herramienta. La asistencia automatizada comprimió el coste de materializarlas, pero no las concibió. La diferencia entre este proyecto y otro de calidad inferior, en presencia de las mismas herramientas, reside exactamente donde ha residido siempre: en la calidad del análisis previo a la primera línea de código. Que un único desarrollador haya podido entregar en unos meses un sistema de este alcance es, precisamente, la prueba de esa amplificación.

En cuanto a los objetivos planteados en el *Capítulo 2*, el objetivo general (digitalizar por completo el flujo de reservas del IUCE con una plataforma trazable, segura y eficiente) se considera alcanzado en su dimensión funcional. Los objetivos específicos del producto se han cubierto en su práctica totalidad, e incluso se ha superado el alcance inicial con un panel de métricas que convierte los datos de operación en información de apoyo a la toma de decisiones sobre los espacios (*véase p. 23, sección 5.8.*), y los objetivos del proceso (aplicación de métodos ágiles adaptados, trazabilidad, aseguramiento de calidad continuo y publicación de releases versionadas) se han cumplido y constituyen, de hecho, la aportación más madura del trabajo. La transparencia sobre el uso de IA generativa, documentada en la *sección 5.11* (*véase p. 26*), forma parte indisociable de esa madurez: un trabajo que se apoya en estas herramientas y no lo declara no es más honesto por ocultarlo, simplemente menos riguroso.

7.2. Líneas de trabajo futuras

El sistema entregado cubre el flujo completo de reservas, pero deja abiertas varias líneas de evolución que se desprenden de forma natural de las decisiones de diseño adoptadas y que un futuro desarrollador podría abordar sin reescribir el sistema.

En el plano del login de la aplicación, la arquitectura de autenticación se diseñó deliberadamente para admitir proveedores adicionales sin tocar el modelo de datos (*véase p. 17, sección 5.2.*). La integración con el SSO de la Universidad de Salamanca permitiría el inicio de sesión único institucional, eliminando el paso del magic link para los usuarios ya autenticados en el dominio universitario. Es la extensión de mayor valor inmediato para los investigadores del IUCE. Sin embargo, requeriría de cooperación entre servicios informáticos (CPD) y el desarrollador para poder llevarlo a cabo.

En el plano funcional, el modelo de reserva recurrente como instancias independientes habilita de forma directa la gestión de excepciones sobre una serie (cancelar o reubicar una única sesión sin afectar al resto), funcionalidad que el modelo de datos ya soporta pero que la interfaz actual no expone por completo.

En el plano de la calidad y la explotación, las líneas más relevantes son la ampliación de la cobertura de pruebas hacia tests de integración de extremo a extremo sobre los flujos completos de solicitud y aprobación, la incorporación de pruebas de carga que validen empíricamente las hipótesis de rendimiento formuladas en la *sección 5.9* (*véase p. 23*), y la evolución de la integración continua hacia un despliegue continuo que reduzca aún más la distancia entre el código aceptado en la rama principal y el sistema en producción.

Finalmente, en el plano de transferencia, la elección de unas herramientas de código abierto y la documentación de arquitectura, API y operación entregada como anexos persiguen un objetivo que excede a este proyecto: que IUCE Reservas pueda adaptarse, con un esfuerzo razonable, a otras unidades académicas con la misma necesidad de gestionar espacios. Esa

posibilidad de reutilización, más que cualquier funcionalidad concreta, es la medida última del valor del trabajo realizado.

8. REFERENCIAS BIBLIOGRÁFICAS

- Allen, J. F. (1983). Maintaining knowledge about temporal intervals. *Communications of the ACM*, 26(11), 832–843. <https://doi.org/10.1145/182.358434>
- Armenteros Cosme, P., Arias González, M., & Ramos Jiménez, M. (2026). *IA Generativa: Aprende a comunicarte con la tecnología. Uso responsable, crítico y aplicado de la IA generativa* [Material didáctico]. Microcredencial en IA Generativa, Grupo de investigación BISITE, Universidad de Salamanca.
- Beck, K., Beedle, M., van Bennekum, A., Cockburn, A., Cunningham, W., Fowler, M., Grenning, J., Highsmith, J., Hunt, A., Jeffries, R., Kern, J., Marick, B., Martin, R. C., Mellor, S., Schwaber, K., Sutherland, J., & Thomas, D. (2001). *Manifesto for agile software development*. <https://agilemanifesto.org/>
- Bonneau, J., Herley, C., van Oorschot, P. C., & Stajano, F. (2012). *The quest to replace passwords: A framework for comparative evaluation of web authentication schemes*. En *2012 IEEE Symposium on Security and Privacy* (pp. 553–567). IEEE. <https://doi.org/10.1109/SP.2012.44>
- Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387. <https://doi.org/10.1145/362384.362685>
- Cohn, M. (2004). *User stories applied: For agile software development*. Addison-Wesley. ISBN 978-0-321-20568-1.
- Comer, D. (1979). The ubiquitous B-tree. *ACM Computing Surveys*, 11(2), 121–137. <https://doi.org/10.1145/356770.356776>
- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2009). *Introduction to algorithms* (3.^a ed.). MIT Press. ISBN 978-0-262-03384-8.
- Date, C. J. (2003). *An introduction to database systems* (8.^a ed.). Addison-Wesley. ISBN 978-0-321-19784-9.
- Fielding, R. T. (2000). *Architectural styles and the design of network-based software architectures* [Tesis doctoral]. Universidad de California. <https://ics.uci.edu/~fielding/pubs/dissertation/top.htm>
- Fielding, R., & Reschke, J. (Eds.). (2014). *Hypertext Transfer Protocol (HTTP/1.1): Message syntax and routing* (RFC 7230). Internet Engineering Task Force. <https://www.rfc-editor.org/info/rfc7230>
- Fowler, M. (2002). *Patterns of enterprise application architecture*. Addison-Wesley. ISBN 978-0-321-12742-6.

- García-Peñalvo, F. J. (2025). Tres escenarios para la IA en educación: del apoyo responsable a la cocreación. *Education in the Knowledge Society*, 26, e32932. <https://doi.org/10.14201/eks.32932>
- García-Peñalvo, F. J., & Vázquez-Ingelmo, A. (2023). What do we mean by GenAI? A systematic mapping of the evolution, trends, and techniques involved in Generative AI. *International Journal of Interactive Multimedia and Artificial Intelligence*, 8(4), 7–16. <https://doi.org/10.9781/ijimai.2023.07.006>
- Gardner, D. (s.f.). *MRBS: Meeting Room Booking System* [Software]. <https://github.com/meeting-room-booking-system/mrbs-code>
- Gerlich, M. (2025). AI Tools in Society: Impacts on cognitive offloading and the future of critical thinking. *Societies*, 15(1), Article 6. <https://doi.org/10.3390/soc15010006>
- Hardt, D. (Ed.). (2012). *The OAuth 2.0 authorization framework* (RFC 6749). Internet Engineering Task Force. <https://www.rfc-editor.org/info/rfc6749>
- Harel, D. (1987). Statecharts: A visual formalism for complex systems. *Science of Computer Programming*, 8(3), 231–274. [https://doi.org/10.1016/0167-6423\(87\)90035-9](https://doi.org/10.1016/0167-6423(87)90035-9)
- Korbel, N. (2020). *Booked Scheduler* [Software]. <https://github.com/effgarces/BookedScheduler>
- LibreBooking. (s.f.). *LibreBooking* [Software]. <https://github.com/LibreBooking/librebooking>
- Naciones Unidas. (2015). *Transformar nuestro mundo: la Agenda 2030 para el Desarrollo Sostenible* (Resolución A/RES/70/1 aprobada por la Asamblea General el 25 de septiembre de 2015). <https://undocs.org/es/A/RES/70/1>
- Schwaber, K., & Sutherland, J. (2020). *The Scrum Guide: The definitive guide to Scrum: The rules of the game*. Scrum.org. <https://scrumguides.org/>
- Sommerville, I. (2016). *Software engineering* (10.^a ed.). Pearson. ISBN 978-0-13-394303-0.

ANEXOS

Los anexos de este Trabajo de Fin de Grado se publican como ficheros independientes en el repositorio del proyecto, con el fin de mantener separada la memoria académica (este documento, que documenta el proceso y las decisiones de diseño) de la documentación operativa (los manuales, que evolucionan con el código). Cada anexo se describe a continuación con su propósito y ubicación. Todos los enlaces apuntan al repositorio público <https://github.com/ShockyDEV/IUCE-Reservas-TFG>, bajo la carpeta docs/manuales.

Anexo A. Plan de Proyecto Software

Documento de planificación como referencia interna del proyecto. Recoge la planificación inicial del trabajo: estimación temporal organizada por sprints y épicas del Product Backlog, estimación de costes considerando el esfuerzo del autor en horas equivalentes y el coste imputable a tecnologías y servicios externos, análisis de viabilidad económica y legal de la solución, y evaluación de los riesgos asociados al desarrollo y al despliegue acompañada de sus medidas de mitigación. Sirve como contrato de alcance del proyecto y como punto de referencia para evaluar las desviaciones entre lo planificado y lo efectivamente entregado al cierre.

Anexo B. Especificación de Requisitos

Especificación formal de los requisitos del sistema, dirigida tanto a la dirección del TFG como a futuros desarrolladores que necesiten contrastar la implementación con el comportamiento esperado. Recoge el catálogo de requisitos funcionales organizados por épica del Product Backlog, el catálogo de requisitos no funcionales (rendimiento, seguridad, mantenibilidad, usabilidad y accesibilidad), el diagrama de casos de uso desglosado por rol de usuario y la descripción detallada de los casos de uso principales. Incluye, además, la traza completa entre cada historia de usuario implementada y el requisito que la justifica, permitiendo verificar que toda funcionalidad entregada responde a una necesidad documentada.

Anexo C. Especificación de diseño

Documento técnico dirigido al personal de mantenimiento del sistema y a futuros desarrolladores que asuman la evolución del proyecto. Describe la arquitectura general de IUCE Reservas: la disposición de capas (presentación con Next.js App Router, lógica de negocio en rutas de API REST, persistencia con Prisma sobre PostgreSQL), los flujos de datos entre frontend y backend, el modelo de datos completo con diagrama entidad-relación y máquina de estados de la reserva, la arquitectura de despliegue (contenedores Docker para la aplicación y la base de datos, Apache como reverse proxy y Let's Encrypt) y las decisiones técnicas que justifican cada elección. Incluye también las instrucciones para reproducir el entorno local y los pasos para promover una nueva versión a producción.

Anexo D. Documentación técnica de programación

Referencia exhaustiva de la superficie de integración del sistema. Enumera los endpoints REST públicos (catálogo de espacios, autenticación NextAuth) y los endpoints REST internos que sustentan las mutaciones (crear reserva, aprobar, rechazar, cancelar, crear bloqueo administrativo). Para cada operación documenta el contrato de entrada y salida con sus tipos TypeScript, los códigos de estado y los errores tipados que devuelve la capa de servicio, las precondiciones de autorización por rol (USER, ADMIN, SUPER_ADMIN) y ejemplos de invocación. Está pensado tanto para terceros que pudieran consumir la API pública del catálogo en el futuro como para auditar el comportamiento del sistema desde una posición externa al código.

Releases y entregables del proyecto

Al cierre de los hitos relevantes del proyecto se han publicado releases versionadas en GitHub. Cada release agrupa los sprints completados hasta la fecha e incluye changelog detallado, capturas de la interfaz, evidencias de los tests y del análisis estático e instrucciones de despliegue local. La primera de estas publicaciones, v0.1.0, recoge el resultado de los cuatro primeros sprints; la v0.2.0 agrupa los sprints 5 a 7; y la v1.0.0 constituye la versión definitiva del sistema, lista para su despliegue en producción, tras integrar los sprints 8 y 9.

El historial completo y los binarios asociados están disponibles en <https://github.com/ShockyDEV/IUCE-Reservas-TFG/releases>.

Anexo E. Documentación de usuario

Guía orientada al personal investigador, profesorado y personal técnico del IUCE que utilizará la plataforma para solicitar y consultar reservas. Cubre, con capturas de pantalla del flujo real, los pasos completos de uso: alta mediante magic link sobre el correo institucional, exploración del catálogo de espacios, creación de una solicitud de reserva (puntual o recurrente), consulta del estado de las solicitudes y cancelación. Se ha redactado con vocabulario no técnico y con un apartado de preguntas frecuentes que recoge las dudas más habituales planteadas durante las pruebas con usuarios reales.

Manual de personal de administración

Manual dirigido al personal autorizado del IUCE que ejerce el rol de administrador del sistema. Describe el panel de aprobación de solicitudes pendientes, la gestión del catálogo de espacios, la creación de bloqueos administrativos sobre franjas concretas, la consulta del registro de auditoría y la asignación de roles a usuarios. Incluye recomendaciones operativas (criterios habituales de aprobación, plazos sugeridos, cómo comunicar un rechazo) y un protocolo de actuación ante incidencias frecuentes, como conflictos de aprobación o solicitudes erróneas.

Anexo F. Anexo de sostenibilización curricular

Anexo de sostenibilización curricular exigido por la normativa de Trabajo de Fin de Grado de la Universidad de Burgos. Recoge una reflexión, de entre 600 y 800 palabras, sobre la relación del proyecto con los Objetivos de Desarrollo Sostenible (ODS) de la Agenda 2030 aprobada por la Asamblea General de las Naciones Unidas (2015). El análisis identifica los objetivos con los que IUCE Reservas mantiene una relación directa (ODS 4, Educación de calidad; ODS 9, Industria, innovación e infraestructura; ODS 12, Producción y consumo responsables; y ODS 16, Paz, justicia e instituciones sólidas) y describe en qué medida el proyecto contribuye a cada uno, centrándose en los efectivamente pertinentes al ámbito y la escala del trabajo.